Project Structure, Interfaces & Non-Functionals

How the codebase is laid out, key Go interfaces, and everything below the waterline.

📁 Project Directory Structure

REPOSITORY LAYOUT

```
estate-agent-platform/
├── cmd/server/
│   └── main.go          # wires everything together
├── internal/
│   ├── config/
│   │   └── config.go      # env-based config
│   ├── db/
│   │   ├── migrations/    # SQL migration files
│   │   ├── queries/       # sqlc .sql query files
│   │   └── db.go          # pgx pool + sqlc generated
│   ├── middleware/
│   │   ├── tenant.go      # resolves + injects tenant
│   │   ├── auth.go        # session → role
│   │   └── logging.go
│   ├── handler/
│   │   ├── public/        # search, property, booking
│   │   ├── seeker/        # dashboard, auth
│   │   ├── agent/         # listings, diary, inbox, analytics
│   │   └── admin/         # moderation, platform analytics
│   ├── service/           # business logic layer
│   │   ├── property.go
│   │   ├── booking.go
│   │   ├── message.go
│   │   └── notification.go
│   ├── storage/
│   │   ├── storage.go     # interface definition
│   │   ├── s3.go          # AWS S3 + compatible (GCS, MinIO)
│   │   └── local.go       # local disk for dev
│   ├── mail/
│   │   ├── mailer.go      # interface definition
│   │   └── smtp.go        # SMTP implementation
│   ├── worker/
│   │   └── reminder.go    # booking reminders background job
│   └── model/             # domain types
```

KEY DESIGN DECISIONS

sqlc for database queries — SQL stays in `.sql` files, sqlc generates type-safe Go structs and query functions. No ORM magic, no reflection at runtime. Queries are readable and reviewable.

Handler → Service → DB layering — Handlers deal only with HTTP (parse request, call service, render template). Services hold business logic and are testable without HTTP. DB layer is generated by sqlc.

Templates live in `web/templates/` — Full pages in named subdirectories. HTMX fragment responses live in `partials/` and return only the swapped element, not a full page. Handlers detect `HX-Request` header and render accordingly.

Single binary deployment — Static assets and templates embedded into the binary with Go's `embed` package. No separate file server

```
├── web/
│   ├── templates/
│   │   ├── layout/        # base.html, agent.html, admin.html
│   │   ├── public/        # search, property, booking pages
│   │   ├── seeker/
│   │   ├── agent/
│   │   ├── admin/
│   │   └── partials/      # HTMX fragment responses
│   └── static/
│       ├── css/app.css    # Bootstrap 5 + overrides
│       └── js/app.js     # htmx.min.js + minimal vanilla JS
├── Dockerfile
├── docker-compose.yml  # local: postgres + minio + app
├── Makefile
└── go.mod
```

needed. One container, one port.

🔌 Key Go Interfaces (swappable implementations)

storage.Storage

```go
type Storage interface {
  // Generate upload URL for b
  PresignPut(ctx, key string,
    ttl time.Duration) (string

  // Public CDN URL for serving
  PublicURL(key string) string

  // Delete on GDPR erasure
  Delete(ctx, key string) erro
}

// Implementations:
// s3.Storage   — AWS S3 / GCS
// local.Storage — dev only (d
```

mail.Mailer

```go
type Mailer interface {
  SendBookingConfirmation(
    ctx context.Context,
    b *model.Booking) error

  SendReminder(
    ctx context.Context,
    b *model.Booking) error

  SendAgentInvite(
    ctx context.Context,
    u *model.User,
    token string) error

  SendSearchAlert(
    ctx context.Context,
    s *model.Seeker,
    props []model.Property) er
}
```

auth.Provider

```go
type Provider interface {
  // Authenticate returns user
  Authenticate(
    ctx context.Context,
    email, password string) (*

  // CreateSession returns coo
  CreateSession(
    ctx context.Context,
    userID uuid.UUID) (string,

  ValidateSession(
    ctx context.Context,
    token string) (*model.Sess

  DestroySession(
    ctx context.Context,
    token string) error
}
// v2: add OAuthProvider wrapp
```

📊 Sortable / Filterable Tables (Bootstrap, no jQuery)

All data tables (agent listings, bookings diary, admin agency list etc.) follow a single pattern:

- Sort column and direction passed as URL query params: `?sort=price&dir=asc`
- Filter inputs use `hx-get` with `hx-trigger="input changed delay:300ms"` targeting `#table-body`
- Server renders only the `<tbody>` partial on HTMX requests (detects `HX-Request` header)
- Column headers are anchor tags with toggled sort direction — no JS required for sorting
- Bootstrap 5 `.table .table-hover .table-striped` classes — consistent across all tables
- Pagination via `?page=N&per_page=25` query params, rendered as Bootstrap pagination component

```
/* Column header with sort toggle */
<th>
  <a hx-get="/dashboard/properties"
     hx-target="#table-body"
     hx-vals='{"sort":"price","dir":"asc"}'>
    Price ↕
  </a>
</th>

/* Filter input */
<input type="text" name="q"
  hx-get="/dashboard/properties"
  hx-trigger="input changed delay:300ms"
  hx-target="#table-body"
  hx-include="[name='sort'],[name='dir']"
  placeholder="Filter listings...">

/* Go handler checks HX-Request header */
if r.Header.Get("HX-Request") != "" {
    // render partials/table-body.html only
} else {
    // render full page
}
```

🛡️ Non-Functional Requirements

Concern	Approach	In MVP?
CSRF protection	CSRF token in all HTML forms via middleware; validated on every POST/PATCH/DELETE. HTMX requests include token via `hx-headers`.	✅ Yes
Rate limiting	Per-IP rate limiting on auth endpoints (login, register, booking). Middleware using a token bucket. Prevents brute force and booking spam.	✅ Yes
GDPR — right to erasure	Soft-delete seeker accounts. On erasure request: anonymise PII in DB (name → "Deleted User", email → hash), delete object storage images, purge messages. Audit log retained.	✅ Yes
GDPR — cookie consent	Cookie banner on first visit. Only strictly necessary cookies (session) set before consent. Analytics/tracking (future) gated behind consent.	✅ Yes
Security headers	Middleware sets: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy`, `Referrer-Policy`. HTTPS enforced via redirect.	✅ Yes

Concern	Approach	In MVP?
Input sanitisation	All template output auto-escaped by Go's `html/template`. Free-text fields (description) stripped of HTML on ingress. No raw HTML rendering.	✅ Yes
Database migrations	`golang-migrate` or `goose` — SQL migration files in `db/migrations/`, run automatically on startup in dev, manually in prod with a pre-deploy step.	✅ Yes
Observability	Structured JSON logging (slog). Request ID middleware. Health check endpoint `/healthz`. Basic Prometheus metrics endpoint `/metrics` (request count, latency, DB pool).	✅ Yes
Testing	Unit tests for service layer (mocked interfaces). Integration tests for handlers using `httptest`. DB tests against a real Postgres instance in CI via Docker. No mocking the DB.	✅ Yes
Accessibility	Bootstrap 5 semantic HTML baseline. ARIA labels on interactive elements. Keyboard navigation for booking slot picker. Target WCAG 2.1 AA.	✅ Yes
Agent verification (AML)	Manual approval by platform admin in v1. Document upload (Companies House number) stored in object storage. Automated verification (Companies House API) deferred to v2.	⚡ Partial
Portal feed syndication	Rightmove / Zoopla ADF integration. Significant operational complexity — deferred to post-MVP.	❌ Deferred
Native mobile apps	Mobile-first responsive web. Native iOS/Android deferred.	❌ Deferred

🚀 Deployment (Cloud-Agnostic)

🐳 Docker

Single-binary Go image (~20MB). Multi-stage Dockerfile. Templates + static assets embedded via `go:embed`.

🐘 PostgreSQL

Any managed Postgres: RDS, Cloud SQL, Supabase, Neon, or self-hosted. Connection string via env var.

☁️ Object Storage

S3-compatible API: AWS S3, GCS (via interop), Cloudflare R2, MinIO. Switched via `STORAGE_PROVIDER` env var.

📧 Email

Any SMTP: SES, SendGrid, Postmark, Mailgun. Configured via env vars. Interface is swappable.

```
# All config via environment variables — 12-factor app
DATABASE_URL=postgres://user:pass@host:5432/db?sslmode=require
SESSION_SECRET=<random-32-bytes>
STORAGE_PROVIDER=s3          # s3 | gcs | local
STORAGE_BUCKET=my-bucket
STORAGE_REGION=eu-west-2
STORAGE_CDN_BASE=https://cdn.example.com
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=<key>
MAIL_FROM=hello@platform.com
```

```
BASE_URL=https://platform.com
PORT=8080
```